Take, for example, the *factorial* function:

Listing E.5 Factorial function, JavaScript

```
function factorial(n) {
  if (n < 0) {
    throw new RangeError('n can not be negative');
  } else if (n === 0) {
    return 1;                                                    #1
  }
  let f = factorial(n-1);                                        #2
  return n * f;
}
```

#1 Base case
#2 Recursive call to `factorial`. We are using an explicit variable to make a point (see below).

A call to factorial with a positive integer `n` will require the creation of `n` stack frames: in figure D.1 we sketched the stack frame for a call to `factorial(3)`, where you can see 3 new stack frames are created. Being the stack area in memory fixed and limited (and small compared to *the heap*), this is a recipe for disaster. Try to call it with a sufficiently large value, and you are going to cause a *segmentation fault*.

A *segmentation fault* aka *access violation* is a failure condition, raised by hardware with memory protection, notifying an operating system that some software has attempted to access a restricted area of memory that had not been assigned to the software.